

Project Architecture & Technical Overview

Mandi Sahayak is an intelligent, scalable queue management and slot-booking ecosystem designed to streamline agricultural procurement at mandi centers. The architecture separates user touchpoints, heavy background tasks, real-time caching, and machine learning predictions into decoupled, high-performance layers.

Complete Technology Stack

Layer / Domain	Technology Selected	Purpose
Frontend	Next.js , Tailwind CSS , shadcn/ui , Lucide Icons	Responsive web interfaces for farmers, mandi officers, and administrators with pre-built accessible UI components.
Client State & Data	Zustand , TanStack Query , React Hook Form	Manages client-side application state, efficient server data synchronization, and robust form validation.
Backend API	Next.js API Route Handlers	Full-stack server endpoints handling request validation, business logic coordination, and core routing.
Relational Database	PostgreSQL + Prisma ORM	Stores immutable, relational data including farmer profiles, crop records, slot bookings, tokens, and user roles.
Caching & State	Redis	Maintains high-speed, in-memory live queue counters, active token positions, and rate-limiting metrics.
Background Jobs	BullMQ	Asynchronously processes non-blocking background tasks like batch notifications, scheduled reports, and queue recalibrations.
ML Prediction Service	Python , FastAPI , scikit-learn , pandas , NumPy	Dedicated microservice running regression models to predict accurate wait times based on historical throughput and active variables.

Layer / Domain	Technology Selected	Purpose
Conversational AI	OpenAI / Groq LLM API	Translates raw mathematical wait times into natural regional languages (Hindi/Marathi) and powers document retrieval (RAG).
Communication Layer	WhatsApp Business API & Indian SMS Gateway	Delivers multi-channel, low-friction automated alerts, token receipts, and status updates directly to farmers' mobile devices.
Storage & DevOps	S3-Compatible Storage, Docker, AWS, GitHub	Handles document and receipt uploads, containerized packaging, version control, and cloud deployment.

End-to-End Workflow

1. User Interaction & Request Initiation - Farmers, mandi officers, and administrators interact with the system via the Web Application, WhatsApp Bot, or SMS Gateway. - Incoming requests are routed through the load balancer into the Next.js backend application.
2. Data Persistence & Relational Mapping - The Next.js backend executes database transactions through Prisma ORM. - Core entities (farmer identity, crop specifications, center capacity, and booking slots) are safely committed and validated within PostgreSQL.
3. Real-Time Queue Management & State Synchronization - Upon successful booking issuance, active token tracking is immediately synchronized into Redis for ultra-low-latency reads. - When center operators update a farmer's physical workflow stage (e.g., transitioning from quality testing to weighing), the live queue status updates instantly across all dashboards.

4. Asynchronous Notification Pipeline

- Job creation payloads are pushed to BullMQ.
- Background workers pick up tasks to dispatch automated status alerts and reminders via WhatsApp and SMS without blocking the main event loop.

5. Machine Learning & Conversational Translation

- The Python FastAPI service continuously ingests operational metrics (active queue length, staff availability, historical processing intervals, and weather parameters).
- Scikit-learn regression models calculate exact estimated waiting times.
- The LLM layer wraps these numerical predictions into natural language responses for instant communication over the WhatsApp interface.